Task 2 - Solution: Closed Knight's Tour

Overview

We solve the closed knight's tour problem using **Warnsdorff's Rule**, a greedy heuristic that efficiently finds knight's tours without exhaustive search.

How Warnsdorff's Rule Works

Core Idea

At each step, move to the adjacent square that has the **fewest available onward moves**. This prioritizes "hard-to-reach" squares early, preventing the knight from getting trapped later.

Step-by-Step Algorithm

```
1. Place the knight on any starting square (r0, c0).
2. Mark the starting square as visited.
3. Repeat n^2 - 1 times:
   a. Look at all unvisited squares reachable by a knight move.
   b. For each candidate, count how many unvisited squares IT can reach (its
"degree").
   c. Move to the candidate with the smallest degree.
   d. Break ties by choosing the square furthest from the board center.
   e. Mark the new square as visited.
4. Check if the final square is one knight move away from (r0, c0).
   - If yes: closed tour found.
   - If no: restart with a different starting position.
```

Why It Works

Warnsdorff's rule is a heuristic, not a guaranteed algorithm. However, it works remarkably well in practice:

- By visiting constrained squares first, it avoids creating isolated regions.
- On boards where closed tours exist (even n >= 6), it almost always succeeds.
- Time complexity is O(n^2) per attempt -- each of the n^2 moves examines at most 8 neighbors.

Handling the "Closed" Requirement

A plain Warnsdorff's tour finds an **open** tour (start and end anywhere). For a **closed** tour:

1. **After completing the tour**, we verify the last square is a knight move from the start.
2. **If not**, we restart the route from a different starting square.

> there exists variations of this algorithm where limited backtracking is used. though however they all line up to have the same complexity.

Tie-Breaking for Closed Tours

for neighbors with equal degrees, we prefer squares further from the center.

Implementation Details

Data Structures

- **Board array** `board[n][n]` : stores the move number (0 = unvisited).
- **Knight move offsets**: 8 possible night moves:

```
{-2,-1}, {-2,1}, {-1,-2}, {-1,2}, {1,-2}, {1,2}, {2,-1}, {2,1}
```

- **Degree function** `getCellDegree(n,pos,board)` : counts unvisited neighbors of (r, c).

Pseudocode

```
function closedKnightTour(n):
    for each starting square (sr, sc):
        clear board
        board[sr][sc] = 1
        r, c = sr, sc

        for move = 2 to n*n:
            candidates = getUnvisitedNeighbors(r, c)
            if candidates is empty:
                break  // stuck, try next start

            sort candidates by (degree ascending, distance-to-center
ascending)
            (r, c) = candidates[0]
            board[r][c] = move

        if move == n*n and isKnightMove(r, c, sr, sc):
            return board  // closed tour found!
```

```
    return "No closed tour exists"
```

Output Examples

```
Test case 3: 8x8 board
expected output: possible path.
[        2        5       36       21       46        7       38       23 ]
[       35       20        3        6       37       22       45        8 ]
[        4        1       42       61       44       47       24       39 ]
[       19       34       53       50       41       62        9       48 ]
[       52       15       64       43       60       49       40       25 ]
[       33       18       51       54       63       28       59       10 ]
[       14       55       16       31       12       57       26       29 ]
[       17       32       13       56       27       30       11       58 ]
Test Case 3 passed
```

Complexity Summary

Per-Attempt Complexity: $O(n^2)$

Each attempt to find a closed knight's tour runs in linear time relative to the number of squares:

- **Outer loop:** n^2 iterations (one per square on the board).
- **Inner operations per iteration:** - Get unvisited neighbors: $O(8) = O(1)$ (a knight has at most 8 moves).
- Calculate degree for each neighbor: $O(8) = O(1)$ per neighbor.
- Sort candidates: $O(8 \log 8) = O(1)$ (constant-size set).
- **Total per move:** $O(1)$
- **Total per attempt:** $O(n^2) \times O(1) = O(n^2)$

Total Complexity (All Attempts): $O(k \cdot n^2)$

where k = number of starting positions tried

- On even boards ≥ 6, Warnsdorff's rule succeeds in very few attempts (typically 1–2).
- For practical purposes, **k is a small constant**, so total time ≈ $O(n^2)$.
- Even on boards where multiple restarts are needed, the constant factor remains small due to the heuristic's high success rate.